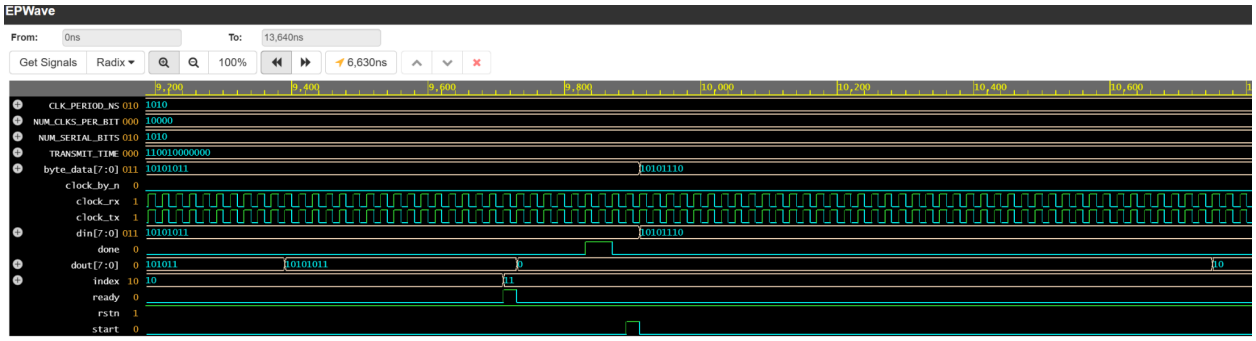
```
# // and may not be used in any way not expressly authorized by Siew.
# //
# Loading sv_std.std
# Loading work.uart_top_testbench(fast)
# Loading work.uart_top(fast)
# Loading work.uart_tx(fast)
# Loading work.uart_rx(fast)
#
# run -all
# Test Passed - Correct Byte Received time=                3070  expected=a5   actual=a5
# Test Passed - Correct Byte Received time=                6430  expected=a8   actual=a8
# Test Passed - Correct Byte Received time=                9710  expected=ab   actual=ab
# Test Passed - Correct Byte Received time=               12990  expected=ae   actual=ae
# ** Note: $stop    : testbench.sv(96)
#    Time: 13650 ns  Iteration: 0  Instance: /uart_top_testbench
# Break in Module uart_top_testbench at testbench.sv line 96
# Stopped at testbench.sv line 96
# exit
# End time: 15:40:14 on Mar 09,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
End time: 15:40:14 on Mar 09,2026, Elapsed time: 0:00:02
*** Summary ********************************************
    qrun: Errors:   0, Warnings:   0
    vlog: Errors:   0, Warnings:   0
    vopt: Errors:   0, Warnings:   1
    vsim: Errors:   0, Warnings:   0
  Totals: Errors:   0, Warnings:   1
Finding VCD file...
./dump.vcd
```

The tests passed when testing with the given testbench

ve opening in a new browser window, set that option on your profile page.



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

We can see the clocks syncing and the done signal being sent shortly after the signal has been received. This shows a successful communication between the modules.
In this simulation we can also se the Baud Rate and Oversampling Accuracy working as minted along with the state transitions and the verification of the data packets that we receive.

Current State	Input / Condition	Next State	Moore Output (tx)
IDLE	tx_start == 0	**IDLE**	1 (Marking)
IDLE	tx_start == 1	**START**	1
START	s_tick count < 15	**START**	0 (Start Bit)
START	s_tick count == 15	**DATA**	0

DATA	s_tick count < 15 OR bit_cnt < 7	**DATA**	data_buffer[bit_cnt]
DATA	s_tick count == 15 AND bit_cnt == 7	**STOP**	data_buffer[7]
STOP	s_tick count < 15	**STOP**	1 (Stop Bit)
STOP	s_tick count == 15	**IDLE**	1 (Triggers tx_done_tick)

Current State	Input / Condition	Next State	Moore Output (rx)
IDLE	rx == 1	IDLE	0
IDLE	rx == 0 (Start edge)	START	0
START	s_tick count < 7	START	0
START	s_tick count == 7	DATA	0 (Sample middle of Start)
DATA	s_tick count < 15 OR bit_cnt < 7	DATA	0
DATA	s_tick count == 15 AND bit_cnt == 7	STOP	0 (Sample middle of Bit 7)
STOP	s_tick count < 15	STOP	0
STOP	s_tick count == 15	IDLE	1 (Data complete)

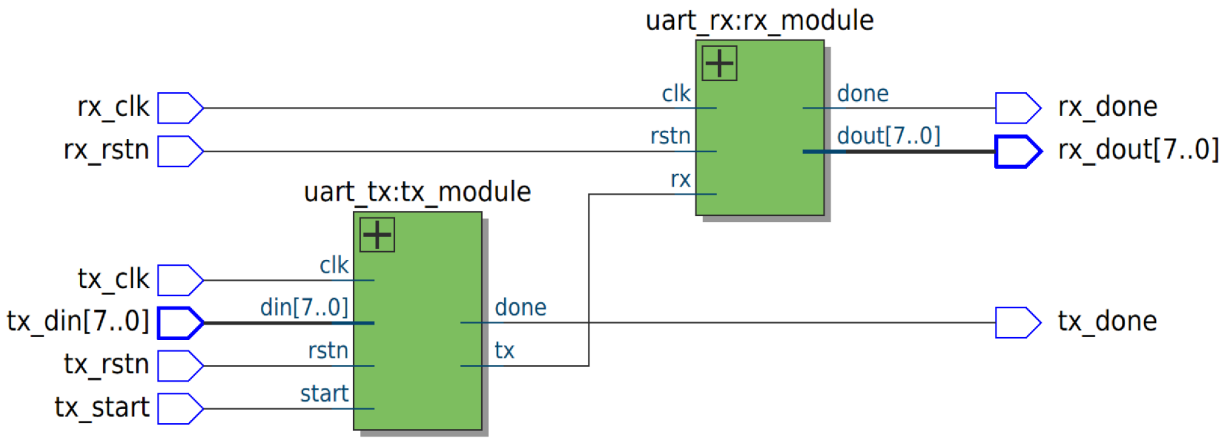
Code (rx):

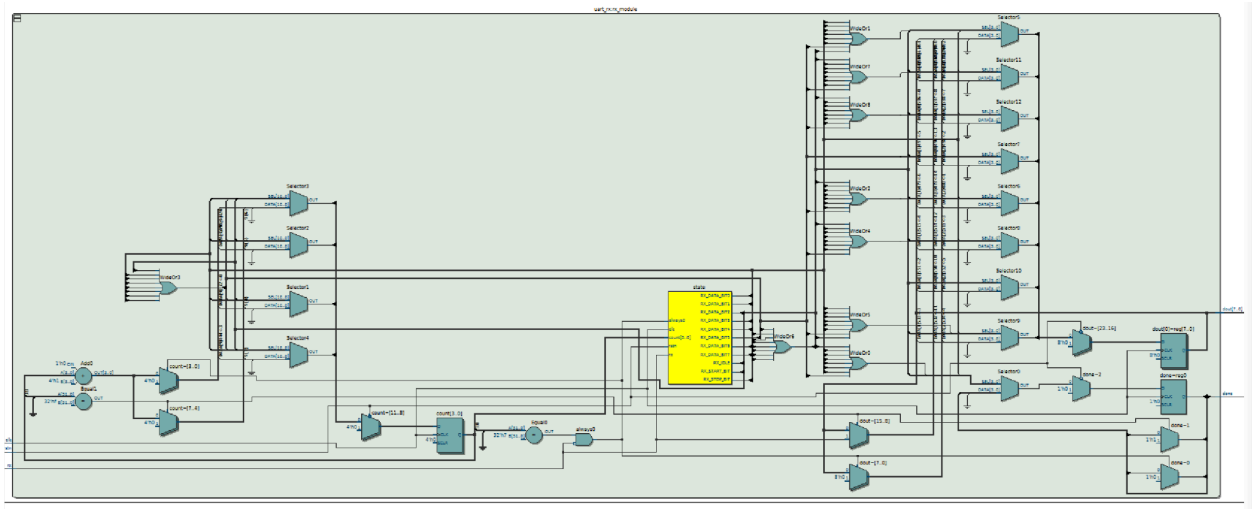
```
// UART RX RTL Code
```

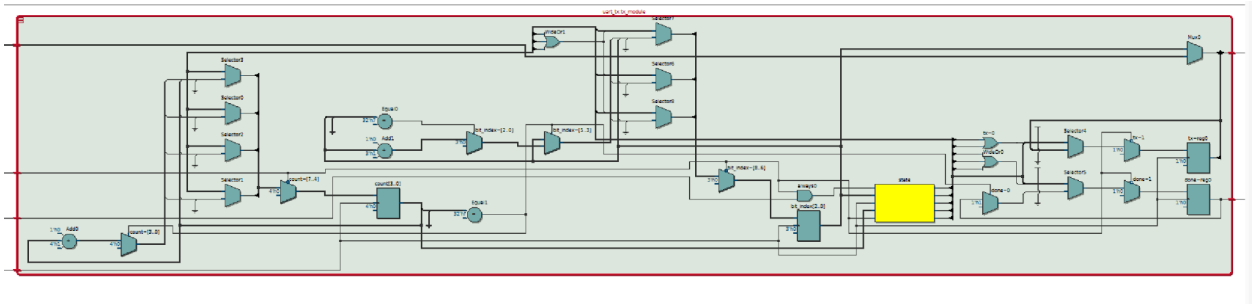
```
module uart_rx #(parameter NUM_CLKS_PER_BIT=16)
(input logic clk, rstn,
 input logic rx, // input serial incoming data
 output logic done, // indicates 8-bit serial data is converted into 8-bit parallel data and available on dout port
 output logic [7:0] dout // 8-bit parallel data output
);

// count variable
logic [$clog2(NUM_CLKS_PER_BIT)-1:0] count;

// state encoding and state variable
enum logic[3:0]{
  RX_IDLE       = 4'b0000,
  RX_START_BIT  = 4'b0001,
  RX_DATA_BIT0  = 4'b0010,
  RX_DATA_BIT1  = 4'b0011,
  RX_DATA_BIT2  = 4'b0100,
  RX_DATA_BIT3  = 4'b0101,
  RX_DATA_BIT4  = 4'b0110,
  RX_DATA_BIT5  = 4'b0111,
  RX_DATA_BIT6  = 4'b1000,
  RX_DATA_BIT7  = 4'b1001,
  RX_STOP_BIT   = 4'b1010} state;


// FSM with single always block for next state,
// present state flipflop and output logic
always_ff@(posedge clk) begin
 if(!rstn) begin
   done <= 0;
   count <= 0;
   dout <= 0;
   state <= RX_IDLE;
 end
 else begin
    case(state)
      RX_IDLE: begin
          done <= 0;
          count <= 0;
          dout <= 0;
          // Wait for rx = 0 indicating start bit
      if(rx == 0) state <= RX_START_BIT;
      else state <= RX_IDLE;
      end
      RX_START_BIT: begin
          // sample start bit value at mid-point, for start bit counter
      // value = 7 is midpoint
          // wait for rx to transition from 1 to 0
          if(rx == 0 && count == ((NUM_CLKS_PER_BIT-1)/2)) begin
              done <= 0;
              state <= RX_DATA_BIT0;
              count <= 0;
              dout <= 0;
          end else begin
              count <= count + 1;
          end
    end
        RX_DATA_BIT0: begin
            // sample start bit value at mid-point
       // for each databit to get midpoint count value is 16
       // counting starts from midpoint of previous bit and ends at midpoint
       // of current data bit
```

```
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[0] <= rx;
            count <= 0;
            state <= RX_DATA_BIT1;
        end else begin
            count <= count + 1;
        end
    end

    RX_DATA_BIT1: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[1] <= rx;
            count <= 0;
            state <= RX_DATA_BIT2;
        end else begin
            count <= count + 1;
        end
    end

    RX_DATA_BIT2: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[2] <= rx;
            count <= 0;
            state <= RX_DATA_BIT3;
        end else begin
            count <= count + 1;
        end
    end

    RX_DATA_BIT3: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[3] <= rx;
            count <= 0;
            state <= RX_DATA_BIT4;
        end else begin
            count <= count + 1;
        end
    end

    RX_DATA_BIT4: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[4] <= rx;
            count <= 0;
            state <= RX_DATA_BIT5;
        end else begin
            count <= count + 1;
        end
    end

    RX_DATA_BIT5: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[5] <= rx;
            count <= 0;
            state <= RX_DATA_BIT6;
        end else begin
            count <= count + 1;
        end
    end

    RX_DATA_BIT6: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[6] <= rx;
```

```
            count <= 0;
            state <= RX_DATA_BIT7;
        end else begin
            count <= count + 1;
        end
    end

    RX_DATA_BIT7: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            dout[7] <= rx;
            count <= 0;
            state <= RX_STOP_BIT;
        end else begin
            count <= count + 1;
        end
    end

    RX_STOP_BIT: begin
        if(count == (NUM_CLKS_PER_BIT-1)) begin
            done <= 1;
            state <= RX_IDLE;
            count <= 0;
        end else begin
            count <= count + 1;
        end
    end


  endcase
 end
end
endmodule: uart_rx
```

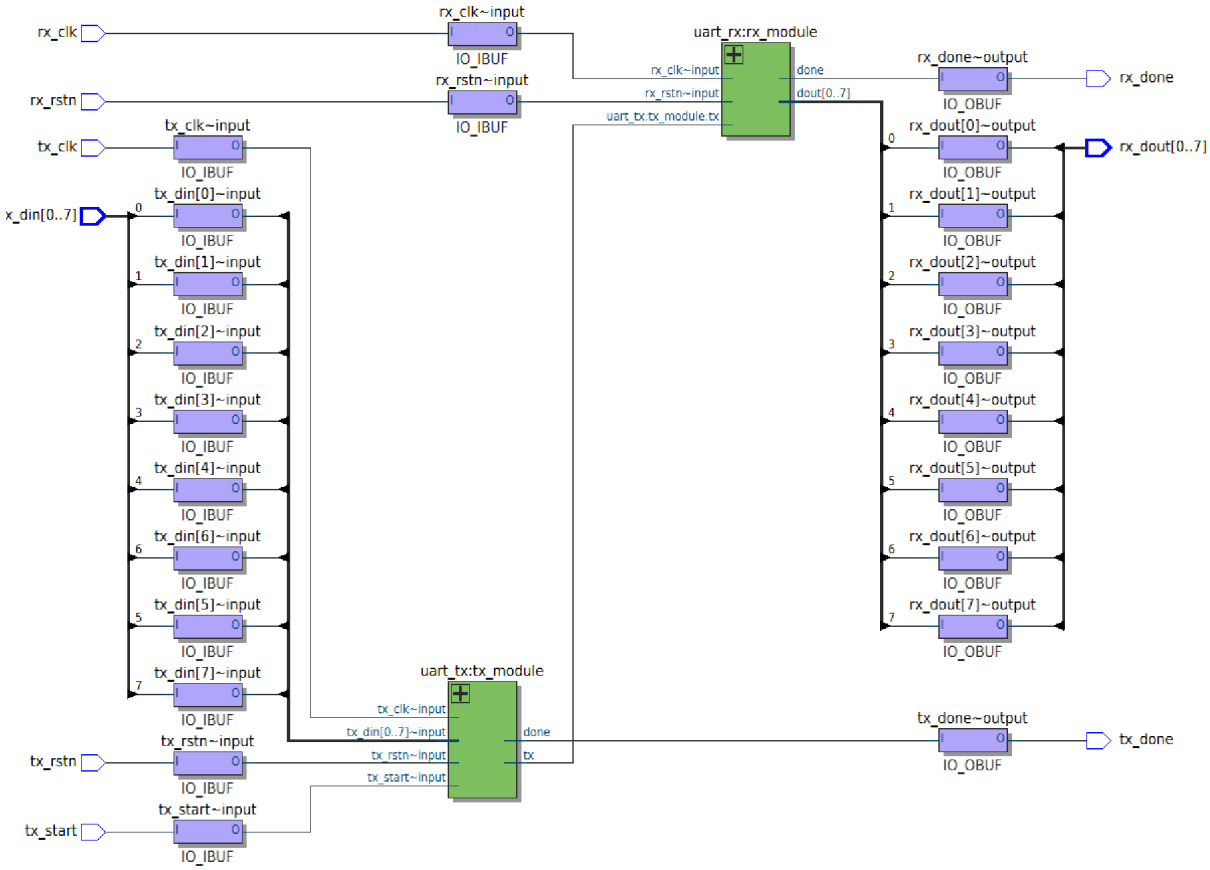

uart_rx:rx_module
rx_clk
rx_rstn
clk
rstn
rx
done
dout[7..0]
rx_done
rx_dout[7..0]
uart_tx:tx_module
tx_clk
tx_din[7..0]
tx_rstn
tx_start
clk
din[7..0]
rstn
start
done
tx
tx_done





	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	38
2		
3	▼ Combinational ALUT usage for logic	54
1	-- 7 input functions	3
2	-- 6 input functions	18
3	-- 5 input functions	12
4	-- 4 input functions	13
5	-- <=3 input functions	8
4		
5	Dedicated logic registers	38
6		
7	I/O pins	23
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	rx_...put
12	Maximum fan-out	24
13	Total fan-out	385
14	Average fan-out	2.79

Shows the logic registers and the ALUT logic usid as wel as pings and total logic utilization


rx_clk
rx_clk~input
IO_IBUF
uart_rx:rx_module
rx_clk~input
done
rx_rstn~input
dout[0..7]
uart_tx:tx_module:tx
rx_done~output
IO_OBUF
rx_done
rx_rstn
rx_rstn~input
IO_IBUF
tx_clk
tx_clk~input
IO_IBUF
x_din[0..7]
tx_din[0]~input
tx_din[1]~input
tx_din[2]~input
tx_din[3]~input
tx_din[4]~input
tx_din[6]~input
tx_din[5]~input
tx_din[7]~input
IO_IBUF
rx_dout[0]~output
rx_dout[1]~output
rx_dout[2]~output
rx_dout[3]~output
rx_dout[4]~output
rx_dout[5]~output
rx_dout[6]~output
rx_dout[7]~output
IO_OBUF
rx_dout[0..7]
uart_tx:tx_module
tx_clk~input
tx_din[0..7]~input
tx_rstn~input
tx_start~input
done
tx
tx_done~output
IO_OBUF
tx_done
tx_rstn
tx_rstn~input
IO_IBUF
tx_start
tx_start~input
IO_IBUF